

POSTGRESQL

Advisory Lock

Concurrency, yarış vəziyyətləri, transaction sərhədləri və EF Core/Npgsql tətbiqi

DƏRİN PRAKTİK BƏLƏDÇİ

Nəzəri əsaslar • SQL funksiyaları • Row lock müqayisəsi

Deadlock • Connection pooling • Monitoring • ECafe analizi

Hazırlanma tarixi: 24 sentyabr 2026

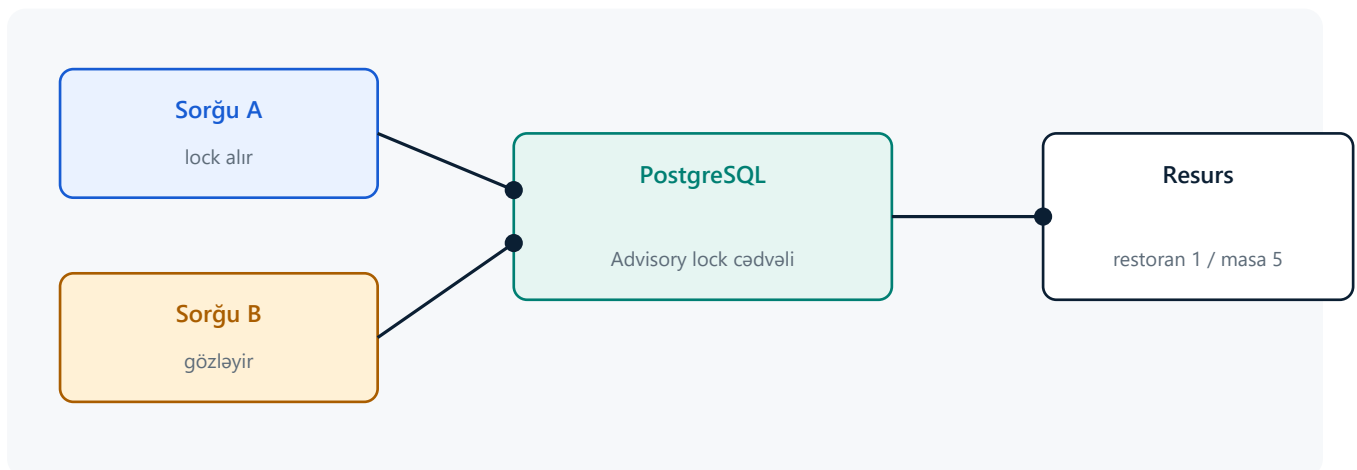
1. Advisory lock nədir?

Əsas ideya: PostgreSQL açarı kilidləyir, açarın mənasını isə tətbiq müəyyən edir.

Advisory lock PostgreSQL daxilində saxlanılan, lakin semantikasi tətbiq tərəfindən razılaşdırılan məntiqi kiliddir. Database bu lock-un “masa 5”, “müşəri 42” və ya “gündəlik hesabat worker-i” olduğunu bilmir. O yalnız eyni rəqəmsal açar üçün zidd lock sorğularını növbələndirir.

Niyə “advisory” adlanır?

Çünki qaydaya əməl etmək məcburiyyətini PostgreSQL yox, tətbiq daşıyır. Kodun bir hissəsi lock alıb, başqa hissəsi eyni resursa lock almadan yazırsa, database bunu avtomatik dayandırmır.



Lock-un özü məlumat sətri deyil. Buna görə mövcud olmayan resursu, gələcək tarix intervalını, xarici servisdəki işi və ya bir neçə cədvələ yayılan biznes invariantını kilidləmək mümkündür.

Bir cümləlik tərif: advisory lock, eyni məntiqi resurs üzərində paralel işləyən əməliyyatlara “bir-bir keçin” qaydası verən database-daxili koordinasiya mexanizmidir.

2. Problem: yarış vəziyyəti

Sadə "əvvəl yoxla, sonra yaz" kodu paralel sorğularda atomik deyil.

Tutaq ki, eyni masa üçün eyni anda iki rezervasiya sorğusu gəlir. Hər sorğu ayrıca "masa boşdurmu?" yoxlaması edir. Aralarında koordinasiya yoxdursa, hər ikisi köhnə vəziyyəti görüb rezervasiya yarada bilər.

Vaxt	Sorğu A	Sorğu B	Nəticə
T1	Masa boşdur? → Bəli		Hələ yazı yoxdur
T2		Masa boşdur? → Bəli	B də eyni snapshot-u görür
T3	Rezervasiya yazır		1-ci qeyd
T4		Rezervasiya yazır	2-ci zidd qeyd

TOCTOU

Bu problem "time-of-check to time-of-use" adlanır: yoxlama ilə istifadə arasında başqa transaction vəziyyəti dəyişir.

Advisory lock olduqda ardıcılıq dəyişir:

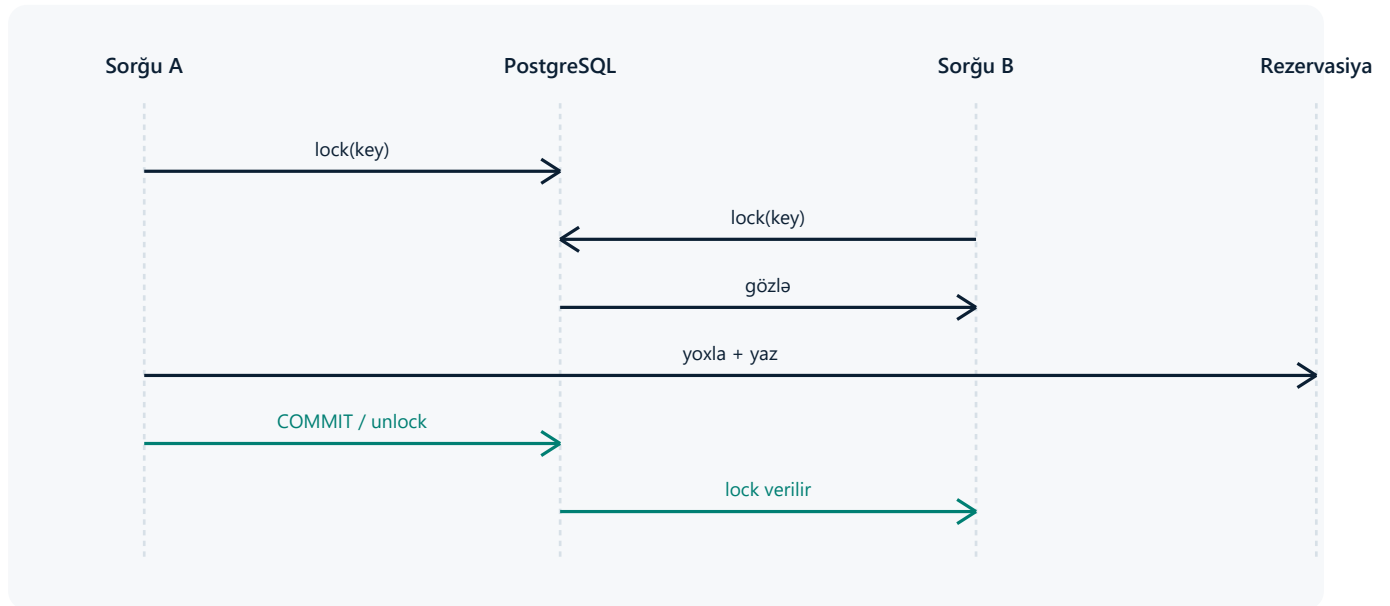
- A eyni resurs açarı üçün lock alır.
- B həmin açarda gözləyir; onun yoxlaması hələ işləmir.
- A yoxlayır, yazır və commit edir.
- Lock B-yə verilir; B artıq A-nın nəticəsini görür və zidd yazını rədd edir.

Lock yoxlamadan əvvəl alınmalıdır

Lock-u yalnız INSERT-dən sonra almaq yarış vəziyyətini həll etmir. Kritik hissə lock → yenidən yoxlama → dəyişiklik → commit ardıcılığıdır.

3. Mental model və həyat dövrü

Advisory lock ayrıca obyekt yox, PostgreSQL lock manager-də saxlanılan açardır.



Transaction-scope lock üçün kritik zona transaction sərhədinin içindədir. Commit və ya rollback baş verən kimi PostgreSQL lock-u avtomatik buraxır.

Mərhələ	Nə baş verir?	Əsas risk
1. Begin	Transaction başlanır	Lock transaction-dan əvvəl alınmamalıdır
2. Acquire	Açar üzrə lock alınır	Səhv açar eyni resursu qorumaz
3. Re-check	Mövcud vəziyyət yenidən oxunur	Lock-dan əvvəlki nəticə köhnəlmiş ola bilər
4. Write	Biznes dəyişikliyi yazılır	Bütün yazılar eyni transaction-da olmalıdır
5. End	Commit/rollback lock-u açır	Uzun transaction uzun gözləmə yaradır

Vacib

Transaction-level advisory lock savepoint rollback zamanı açılmaz; bütün transaction bitənə qədər qalır. Bu, kritik zonanın təsadüfən erkən açılmasının qarşısını alır.

4. Session və transaction lock

Ən vacib seçim lock-un nə vaxt avtomatik buraxılmasıdır.

Xüsusiyyət	Session-level	Transaction-level
Funksiya	<code>pg_advisory_lock</code>	<code>pg_advisory_xact_lock</code>
Müddət	Session bağlanana və ya <code>unlock</code> çağırılana qədər	Transaction commit/rollback edənədək
Rollback	Lock-u açmır	Avtomatik açır
Əl ilə unlock	Mümkündür	Mümkün deyil
Təkrar acquire	Sayğac artır; eyni sayda unlock lazımdır	Eyni transaction-da əlavə təsir yaratmır
Connection pool	Yüksək risk: lock pool-a qayıda bilər	Request-scope üçün təhlükəsiz seçim
Tipik istifadə	Uzunömürlü liderlik, xüsusi daemon	API əməliyyatı, rezervasiya, ödəniş

Default seçim

Web API və EF Core əməliyyatlarında adətən `pg_advisory_xact_lock` seçilir. Həyat dövrü transaction-la eyni olduğundan exception, cancellation və rollback zamanı lock sızması azalır.

Session lock nə vaxt uyğundur?

Yalnız session-un ömrünü özünüz idarə etdiyiniz, connection pooling davranışını tam bildiyiniz və explicit unlock-u finally blokunda zəmanətə aldığınız xüsusi hallarda.

5. Funksiyaların tam xəritəsi

Exclusive/shared və blocking/try variantları bir-birindən ayrılır.

Funksiya ailəsi	Scope	Rejim	Davranış
pg_advisory_lock	Session	Exclusive	Lock boşalanadək gözləyir
pg_try_advisory_lock	Session	Exclusive	Dərhal true/false qaytarır
pg_advisory_lock_shared	Session	Shared	Digər shared lock-larla yanaşı işləyir
pg_try_advisory_lock_shared	Session	Shared	Gözləmədən true/false
pg_advisory_unlock	Session	Exclusive	Bir acquire səviyyəsini açır
pg_advisory_unlock_shared	Session	Shared	Shared lock-u açır
pg_advisory_unlock_all	Session	Hamısı	Cari session-un bütün advisory lock-larını açır
pg_advisory_xact_lock	Transaction	Exclusive	Transaction bitənədək saxlayır
pg_try_advisory_xact_lock	Transaction	Exclusive	Dərhal true/false
pg_advisory_xact_lock_shared	Transaction	Shared	Oxşar shared işlər paralel ola bilər
pg_try_advisory_xact_lock_shared	Transaction	Shared	Gözləmədən true/false

Exclusive lock eyni aşardakı shared və exclusive lock-larla ziddir. Shared lock başqa shared lock-larla paralel işləyir, exclusive lock-u isə gözlədir.

Blocking variant növbəyə dayanır. Try variant gözləmir; işi təxirə salmaq, 409/423 qaytarmaq və ya başqa worker-a buraxmaq üçün əlverişlidir.

6. Açar dizaynı

Lock düzgün açar seçimi qədər güclüdür.

PostgreSQL iki ayrı açar formasını qəbul edir: bir 64-bit bigint, yaxud iki 32-bit integer. Bu iki açar sahəsi bir-biri ilə üst-üstə düşmür.

```
-- Bir bigint açar
SELECT pg_advisory_xact_lock(900000000005);

-- İki integer açar
SELECT pg_advisory_xact_lock(1, 5);
```

Strategiya	Nümunə	Qiymətləndirmə
(namespace, id)	(TABLE_RESERVATION, tableId)	Oxunaqlı və collision riski az
(tenant, resource)	(restaurantId, tableId)	Multi-tenant sistem üçün praktik
Deterministik 64-bit hash	hash(restaurant/table/date)	Mürəkkəb composite açar; collision analizi lazımdır
Runtime GetHashCode	string.GetHashCode()	İstifadə etməyin; proseslər arasında sabit olmaya bilər

Namespace qaydası

Fərqli resurs tiplərinin təsadüfən eyni açarda toqquşmaması üçün açar istehsalını mərkəzləşdirin. Məsələn, 10 = masa rezervasiyası, 20 = müştəri/gün limiti, 30 = background job.

Açar məlumatı gizli deyil

Advisory lock açarı təhlükəsizlik sərhədi deyil. Onu authorization əvəzi kimi istifadə etməyin; access control ayrıca yoxlanmalıdır.

7. ECafe-də hazırkı tətbiq

Kod bazasında transaction-scope iki kilid ailəsi istifadə olunur.

ECafe rezervasiya yaradarkən əvvəl ReadCommitted transaction başlayır, sonra müştəri və masa üçün advisory lock alır. Hər lock-dan sonra biznes vəziyyəti yenidən yoxlanılır.

```
// Müştəri/restoran üçün
SELECT pg_advisory_xact_lock(restaurantId, -customerUserId);

// Masa/restoran üçün
SELECT pg_advisory_xact_lock(restaurantId, tableId);
```

Açar	Qoruduğu invariant	Mövcud davranış
(restaurantId, -customerId)	Müştərinin həmin restoranda lokal gün üzrə bir aktiv rezervasiyası	Eyni müştərinin fərqli gün sorğuları da qısa müddətə növbələnir
(restaurantId, tableId)	Eyni masaya zidd rezervasiyaların yaranmaması	Eyni masanın fərqli vaxt sorğuları da növbələnir

Niyə mənfi customerId?

Kodda ikinci açarın mənfi olması müştəri lock namespace-ni müsbət tableId-lərdən ayırır. ID-lər həmişə müsbət qaldığı müddətdə sadə və işlək konvensiyadır.

Optimallaşdırma imkanı

Hazırkı açarlar təhlükəsiz, amma coarse-grained-dir. Trafik böyüsə customer lock-a lokal gün, table lock-a rezervasiya intervalı daxil edilə bilər. Bu dəyişiklik yalnız mərkəzləşdirilmiş, deterministik key factory və collision testləri ilə edilməlidir.

8. ECafe rezervasiya ardıcılığı

Düzgünlük lock-un özündən çox əməliyyat sırasından gəlir.

#	Addım	Niyə bu yerdədir?
1	Restoran, vaxt və masa girişləri ilkin yoxlanılır	Aydın səhvləri transaction-dan əvvəl rədd edir
2	ReadCommitted transaction başlayır	Bütün kritik iş vahidini bir sərhədə salır
3	Customer lock alınır	Gündəlik limit yoxlamasını serial edir
4	Həmin gün aktiv rezervasiya yenidən yoxlanılır	Paralel sorğunun nəticəsini görür
5	Table lock alınır	Eyni masa üzrə əməliyyatları serial edir
6	Masa availability yenidən yoxlanılır	Double-booking-i dayandırır
7	Reservation + history yazılır	Audit izi eyni transaction-dadır
8	Commit	Dəyişiklik görünür və hər iki lock açılır

Sabit lock sırası

Bütün kod yolları əvvəl customer, sonra table lock almalıdır. Başqa endpoint əvvəl table, sonra customer alarsa, qarşılıqlı gözləmə və deadlock yarana bilər.

Bu yanaşmada iki eyni sorğudan biri lock-u alır, digəri gözləyir. Birinci commit etdikdən sonra ikinci sorğu artıq gündəlik limit və ya masa doluluğunu görərək biznes xətası qaytarır.

9. Read Committed niyə kifayətdir?

Isolation level və advisory lock fərqli problemləri həll edir.

Read Committed-də hər SQL statement başladığı anda commit olunmuş vəziyyəti görür. Təkbəşinə bu isolation “yoxla, sonra yaz” yarışını aradan qaldırmır. Lakin ikinci sorğunun yoxlaması advisory lock gözləməsindən sonra işə düşürsə, birinci transaction-ın commit etdiyi vəziyyəti görür.

Mexanizm	Nəyi təmin edir?	Nəyi təmin etmir?
Read Committed	Dirty read yoxdur; hər statement təzə commit-ləri görə bilər	Bir neçə statement birlikdə avtomatik serial olmur
Advisory lock	Razılaşdırılmış açar üzrə əməliyyatları növbələyir	Lock almayan kodu dayandırır
İkisi birlikdə	Lock-dan sonrakı re-check son commit-i görür	Səhv açar və səhv sıralama hələ riskdir

Səhv nümunə

Availability nəticəsini transaction-dan əvvəl hesablayıb, lock-dan sonra həmin köhnə boolean-a güvənmək olmaz. Lock-dan sonra database sorğusu təkrar icra edilməlidir.

Serializable isolation daha geniş avtomatik qoruma verə bilər, amma serialization failure və retry tələb edir. Məhdud, aydın biznes invariantında advisory lock daha məqsədli və proqnozlaşdırıla bilən ola bilər.

10. Row lock ilə fərq

FOR UPDATE yalnız artıq mövcud olan konkret sətirləri kilidləyir.

Meyar	Row lock (FOR UPDATE)	Advisory lock
Nə kilidlənir?	Mövcud table row	Tətbiqin seçdiyi rəqəmsal açar
Olmayan resurs	Kilidlənəcək row yoxdur	Gələcək slot və ya virtual resurs kilidlənə bilər
Tətbiq məcburiyyəti	Eyni row-a yazan DB əməliyyatları lock-a tabedir	Bütün kod yolları konvensiyaya əməl etməlidir
Plain SELECT	Adətən bloklanmır	Lock-a müraciət etməyən SELECT bloklanmır
İstifadə	Balans, status row-u, mövcud aggregate	Rezervasiya slotu, singleton job, cross-table invariant

Məsələn, masa sətirini FOR UPDATE kilidləmək eyni masanın bütün tarixlərini bloklaya bilər və table row-u mövcuddur. Advisory lock isə açara tarix intervalı əlavə edilərsə yalnız müəyyən slotu koordinasiya edə bilər.

Birlikdə istifadə oluna bilər

Advisory lock biznes əməliyyatını serial edir, row lock isə dəyişdirilən konkret aggregate-in itirilmiş update problemini qoruyur. Bunlar rəqib deyil; fərqli qatlarda tamamlayıcıdır.

Row lock-lar da transaction sonuna qədər saxlanır və deadlock yarada bilər. Buna görə lock sırası hər iki mexanizm üçün sənədləşdirilməlidir.

11. Constraint, isolation və distributed lock

Advisory lock hər problemin yeganə həlli deyil.

Alət	Güclü tərəf	Məhdudiyyət	Seçim signalı
UNIQUE / EXCLUDE	Database səviyyəsində sərt invariant	Yalnız modelləşdirilə bilən məlumat forması	Dublikat heç vaxt qəbul edilə bilməz
Row lock	Mövcud row-a təbii bağlılıq	Olmayan slotu qorumur	Aggregate row mövcuddur
Advisory lock	Virtual/cross-table resurs	Könüllü konvensiya	Aydın biznes açarı var
SERIALIZABLE	Geniş anomaliya qoruması	Retry və daha yüksək xərc	Invariant mürəkkəbdir
Redis/distributed lock	Bir neçə storage və sistem	Lease, clock, fencing mürəkkəbliyi	Kritik iş DB transaction sərhədindən kənardadır

Defense in depth

Mümkün olduqda advisory lock-u database constraint ilə tamamlayın. Lock istifadə edən kod səhvən bypass edilsə belə constraint son müdafiə xətti olar.

ECafe-də "müşəri/restoran/lokal gün üzrə maksimum bir aktiv rezervasiya" status və vaxt pəncərəsindən asılıdır. Sadə UNIQUE indeks bunu tam ifadə etmir; advisory lock burada biznes invariantını serial etmək üçün məntiqlidir.

Masa intervalının üst-üstə düşməsinə PostgreSQL range + exclusion constraint ilə modelləşdirmək mümkündür, lakin statusun availability-yə təsiri və dəyişən iş saatları dizaynı mürəkkəbləşdirir. Advisory lock yenə faydalı ola bilər.

12. Connection pooling təhlükəsi

Session lock ilə pooled connection ən çox rast gəlinən istehsal səhvlərindəndir.

Npgsql pool connection-u request bitəndə fiziki olaraq bağlamaya bilər; onu növbəti request üçün pool-a qaytarır. Session-level advisory lock explicit açılmayıbsa, həmin fiziki PostgreSQL session-da qala bilər.

Hadisə	Nəticə
Request A session lock alır	Lock backend PID-yə bağlanır
Kod exception verir, unlock çağırılır	Transaction rollback olsa belə session lock qalır
Connection pool-a qaytarılır	Fiziki session yaşamaya davam edir
Request B həmin connection-u alır	Əlaqəsiz request lock-u miras ala bilər
Digər connection-lar eyni açarda gözləyir	İzahı çətin istehsal donması yaranır

Qayda

Request-scoped database işi üçün transaction-level lock seçin. Session-level lock məcburidirsə, eyni açar üçün acquire/unlock sayını izləyin və unlock-u finally blokunda edin.

Transaction-level lock-un üstünlüyü odur ki, connection pool-a qayıtmazdan əvvəl commit və ya rollback lock-u PostgreSQL səviyyəsində buraxır. Bu, tətbiq kodundakı bir çox cleanup səhvinə qarşı daha etibarlıdır.

13. Deadlock və lock sırası

Advisory lock deadlock-dan immun deyil.

A transaction-u əvvəl K1, sonra K2; B transaction-u əvvəl K2, sonra K1 alırsa, hər biri digərini gözləyə bilər. PostgreSQL deadlock-u aşkarlayır və transaction-lardan birini ləğv edir.

```
-- Transaction A          -- Transaction B
SELECT lock(K1);           SELECT lock(K2);
SELECT lock(K2); -- gözləyir SELECT lock(K1); -- gözləyir
```

Qarşısını alma qaydaları:

- Bütün endpoint və worker-lar üçün global lock sırası təyin edin.
- Birdən çox eyni tip açarı həmişə sort olunmuş qaydada alın.
- Transaction daxilində şəbəkə çağırışı, fayl upload-u və uzun hesablamadan qaçın.
- Deadlock və serialization failure üçün məhdud, jitter-li retry tətbiq edin.
- Lock wait vaxtını ölçün; gözləmə biznes latency-sinin bir hissəsidir.

ECafe üçün tövsiyə edilən sıra

1) customer/day lock, 2) table/interval lock, 3) konkret reservation/payment row lock. Bu sıra bütün write endpoint-lərdə eyni qalmalıdır.

14. Timeout və fail-fast davranışı

Sonsuz gözləmə texniki baxımdan mümkün, məhsul baxımından arzuolunmazdır.

```
BEGIN;  
SET LOCAL lock_timeout = '2s';  
SELECT pg_advisory_xact_lock(1, 5);  
-- kritik yoxlama və yazılar  
COMMIT;
```

lock_timeout yalnız lock gözləməsinə limit qoyur. statement_timeout bütün statement müddətini məhdudlaşdırır. LOCAL istifadə edildikdə parametr yalnız cari transaction-a aiddir.

Yanaşma	UX	Uyğun ssenari
Blocking lock	Qısa gözləmə, sonra nəticə	Yazının tez bitdiyi rezervasiya əməliyyatı
Try lock	Dərhal "hazırda emal olunur"	Worker, idempotent job, fail-fast API
Timeout	Məhdud gözləmə, kontrollu xəta	İstehsalda request latency sərhədi

Xəta xəritəsi

Lock timeout-u 500 kimi gizlətməyin. Məhsul qərarına uyğun 409 Conflict, 423 Locked və ya retryable texniki xəta qaytarın; traceId və lock resource tipi log-da olsun, xam açarı isə ehtiyac yoxdursa istifadəçiyə göstərməyin.

15. Monitoring və diagnostika

Advisory lock-lar pg_locks görünüşündə izlənə bilər.

```
SELECT
  a.pid, a.username, a.state, a.query_start,
  l.locktype, l.mode, l.granted,
  l.classid, l.objid, l.objsubid,
  pg_blocking_pids(a.pid) AS blocked_by,
  left(a.query, 120) AS query
FROM pg_stat_activity a
JOIN pg_locks l ON l.pid = a.pid
WHERE l.locktype = 'advisory'
ORDER BY l.granted, a.query_start;
```

Sahə	Mənası
granted = true	Lock artıq bu session-a verilib
granted = false	Session lock növbəsində gözləyir
mode	ExclusiveLock və ya ShareLock
pid	pg_stat_activity ilə birləşdirilən backend prosesi
pg_blocking_pids	Birbaşa və ya növbə səbəbindən bloklayan PID-lər
classid / objid	Advisory açarın pg_locks daxilində parçalanmış təsviri

İstehsal metrikləri

Lock wait duration, timeout sayı, deadlock sayı, açar namespace-i, endpoint adı və transaction müddətini ölçün. Yalnız error log-u contention-in tədricən artmasını göstərməyə bilər.

16. EF Core / Npgsql nümunəsi

Bütün əməliyyat eyni DbContext, connection və transaction üzərində qalmalıdır.

```
await using var tx = await db.Database.BeginTransactionAsync(
    IsolationLevel.ReadCommitted, cancellationToken);

await db.Database.ExecuteSqlInterpolatedAsync(
    $"SELECT pg_advisory_xact_lock({restaurantId}, {tableId})",
    cancellationToken);

var isAvailable = await CheckAvailabilityAsync(
    restaurantId, tableId, reservedAt, cancellationToken);

if (!isAvailable)
    throw new BusinessException("TableUnavailable");

db.Reservations.Add(reservation);
await db.SaveChangesAsync(cancellationToken);
await tx.CommitAsync(cancellationToken);
```

Yuxarıdakı nümunədə əsas xüsusiyyətlər:

- Interpolated SQL parametrləşdirilir; açar string birləşdirmə ilə qurulmur.
- Lock transaction başladıqdan sonra alınır.
- Availability lock-dan sonra yenidən soruşulur.
- SaveChanges və commit eyni DbContext transaction-ındadır.
- Exception zamanı await using dispose/rollback edir və xact lock açılır.

Diqqət

Transaction içində notification HTTP çağırışı və ya MinIO upload-u saxlamaq lock müddətini uzada bilər. Database commit-dən sonra outbox worker ilə xarici side effect göndərmək daha dayanıqlıdır.

17. Anti-patternlər

Ən təhlükəli səhvlər ilk baxışda işləyən kodda gizlənir.

Anti-pattern	Nəticə	Düzəliş
Lock-dan əvvəlki nəticəyə güvənmək	Köhnə snapshot ilə zidd yazı	Lock-dan sonra re-check
Bəzi code path-lərdə lock-u atlamaq	Konvensiya pozulur	Write girişlərini bir service-də mərkəzləşdir
Session lock + pool	Sızmış lock və anlaşılmaz bloklama	xact lock və ya guaranteed unlock
Random / GetHashCode açarı	Proseslər eyni resursu fərqli açara çevirir	Sabit key factory
Fərqli lock sırası	Deadlock	Qlobal sıra və sort
Uzun transaction	Queue, timeout, throughput itkisi	Kritik zonanı qısalt
LIMIT ilə lock funksiyasını təhlükəli yazmaq	Planner gözlənilməz çox lock ala bilər	Əvvəl subquery-də limit et
Advisory lock-u auth saymaq	Məlumat sızması / icazəsiz əməliyyat	Authorization ayrıca

```
-- Təhlükəli ifadə qiymətləndirməsi
SELECT pg_advisory_lock(id) FROM jobs WHERE status = 'ready' LIMIT 10;

-- Daha aydın sərhəd
SELECT pg_advisory_lock(id)
FROM (SELECT id FROM jobs WHERE status = 'ready' LIMIT 10) q;
```

Shared memory limiti

Advisory və regular lock-lar PostgreSQL-in shared lock pool-undan istifadə edir. Çox böyük sayda fərqli açarı uzun müddət saxlamaq `max_locks_per_transaction` və `max_connections` ilə bağlı resurs təzyiqi yarada bilər.

18. Concurrency test planı

Test yalnız "iki request göndərdim" deyil; nəticə və gözləmə davranışı da sübut edilməlidir.

Test	İcra	Gözlənilən nəticə
Eyni masa / eyni vaxt	20 paralel create request	Yalnız 1 uğurlu; qalanları business conflict
Eyni müştəri / eyni lokal gün	Fərqli masalara paralel request	Yalnız 1 aktiv rezervasiya
Eyni masa / fərqli vaxt	Paralel request	Düzgünlük qorunur; coarse lock səbəbilə qısa növbə ola bilər
Fərqli masa	Paralel request	Müstəqil lock açarları; paralel tamamlanma
Rollback	Lock-dan sonra intentional exception	Lock avtomatik açılır; sonrakı request davam edir
Cancellation	Gözləyən request cancel edilir	Connection sağlam qalır, transaction rollback olur
Deadlock	Tərs lock sırası simulyasiya edilir	Bir transaction abort; retry policy işləyir
Timeout	Lock 2 saniyədən çox saxlanır	Kontrollu retryable/409/423 xəta

Test zamanı aşağıdakıları ayrıca təsdiqləyin:

- Database-də zidd row sayı həqiqətən birdir; yalnız HTTP cavablarına baxmayın.
- pg_locks-də gözləyən lock görünür və commit-dən sonra yox olur.
- Metriklərdə lock wait və transaction duration ölçülür.
- Timezone sərhədi: gecə 00:00 ətrafında lokal gün açarı və day range düzgündür.

19. Dizayn checklist-i

Yeni advisory lock əlavə etməzdən əvvəl bu sualların hamısına cavab verin.

☐ Qorunan biznes invariantı bir cümlə ilə yazılıbmı?	☐ Eyni resurs bütün proseslərdə eyni deterministik açara çevrilirmi?
☐ Namespace toqquşması mümkündürmü?	☐ Lock transaction başladıqdan sonra və yoxlamadan əvvəl alınırımı?
☐ Lock-dan sonra vəziyyət database-dən yenidən oxunurmu?	☐ Bütün write code path-ləri eyni lock qaydasına əməl edirmi?
☐ Çoxlu lock varsa global sıra müəyyən edilibmi?	☐ Timeout, cancellation və deadlock retry siyasəti varmı?
☐ Transaction daxilində xarici şəbəkə çağırışı qalıbmı?	☐ Database constraint ilə əlavə müdafiə mümkündürmü?
☐ pg_locks və tətbiq metrikləri ilə müşahidə edilirmi?	☐ Paralel integrasiya testi və rollback testi yazılıbmı?

ECafe üçün yekun hökm

Hazırkı pg_advisory_xact_lock seçimi doğru istiqamətdir: request-scope, rollback-safe və ReadCommitted ilə uyğun işləyir. Növbəti yetkinlik addımları mərkəzi key factory, lock timeout, contention metrikləri və bütün flow-lar üzrə sabit lock sırasıdır.

20. Terminlər lüğəti

Sənəddə işlənən əsas concurrency anlayışları.

Termin	Sadə izah
Atomicity	Əməliyyatın ya tam baş verməsi, ya da heç baş verməməsi
Concurrency	Birdən çox işin zaman baxımından üst-üstə düşməsi
Race condition	Nəticənin sorğuların təsadüfi icra sırasından asılı olması
Critical section	Eyni resurs üçün eyni anda yalnız məhdud sayda işin girməli olduğu kod
Invariant	Sistemdə həmişə doğru qalmalı biznes qaydası
Lock scope	Lock-un hansı həyat dövrünə bağlı olması: session və ya transaction
Exclusive	Başqa shared və exclusive girişləri bloklayan lock
Shared	Digər shared girişlərə icazə verib exclusive girişi bloklayan lock
Contention	Bir neçə işin eyni lock üçün rəqabət aparması
Deadlock	İşlərin bir-birinin saxladığı lock-u qarşılıqlı gözləməsi
Idempotency	Eyni əməliyyat təkrar çağırılrsa da əlavə yan təsir yaratmaması
Connection pool	Fiziki DB bağlantılarının request-lər arasında təkrar istifadəsi

Yadda saxla

Advisory lock məlumatı "qorumur"; düzgün tərtib edilmiş kritik əməliyyatı növbələndirir. Məlumat təhlükəsizliyi üçün authorization, invariant üçün constraint və müşahidə üçün telemetry yenə ayrıca lazımdır.

21. Rəsmi mənbələr

Mövzunun texniki davranışı PostgreSQL-in rəsmi sənədlərinə əsaslanır.

1. Explicit Locking — Advisory Locks

<https://www.postgresql.org/docs/current/explicit-locking.html#ADVISORY-LOCKS>

Session və transaction scope, rollback davranışı, shared memory limiti və LIMIT ilə bağlı xəbərdarlıq.

2. System Administration Functions — Advisory Lock Functions

<https://www.postgresql.org/docs/current/functions-admin.html#FUNCTIONS-ADVISORY-LOCKS>

pg_advisory_lock, pg_try_advisory_lock, shared və transaction variantlarının rəsmi siyahısı.

3. The pg_locks View

<https://www.postgresql.org/docs/current/view-pg-locks.html>

Lock mode, granted vəziyyəti, PID və advisory açarların monitorinqi.

4. System Information Functions

<https://www.postgresql.org/docs/current/functions-info.html>

pg_blocking_pids funksiyası ilə bloklayan backend-lərin tapılması.

ECafe kod istinadları

- ReservationRepository.AcquireCustomerReservationLockAsync
- TableRepository.AcquireReservationLockAsync
- ReservationManager.CreateReservationAsync

Son fikir

Lock mexanizminin keyfiyyəti yalnız SQL funksiyasına deyil, açarın sabitliyinə, transaction sərhədinə, re-check qaydasına, lock sırasına və müşahidə oluna bilməsinə bağlıdır.